PART 1: NUMBER SYSTEMS

SIGNED BINARY NUMBERS

SAAD BAIG – DIGITAL LOGIC DESIGN

SUBTRACTION WITH COMPLEMENTS

- In the previous lecture, we were adding a negative sign to negative numbers:

M =		03250
N =		72532
10's complement of N =	+	27468
Sum =		30718
(10's compliment of 30718) =	–	69282

Y =		1000011
2's complement of X =	+	0101100
Sum =		1101111
– (2's compliment of Sum) =	–	0010001

- However this negative sign does not exist in binary operations in computers. Because of hardware limitations, computers must represent everything with binary digits.

SIGNED BINARY NUMBERS

- In binary operations, we either have a 0 or a 1, how will the system know whether the number is positive or negative?
- Here we can introduce a way to represent negative or positive numbers. We can use a bit to denote a sign of the number.
  - If the left most digit is a 0, it is a positive number.
  - If the left most digit is a 1, it is a negative number.

SIGNED BINARY NUMBERS

- Example: Demonstrate $(-9)_{10}$ as signed binary numbers.

5-bit Representation

- $(9)_{10}$ in binary = 1001
- $(+9)_{10}$ in binary = 01001
- Signed magnitude representation → 11001
- 1's complement representation → 10110
- 2's complement representation → 10111

8-bit Representation

- $(9)_{10}$ in binary (8-bit) = 00001001
- $(+9)_{10}$ in binary (8-bit) = 00001001
- Signed magnitude representation → 10001001
- 1's complement representation → 11110110
- 2's complement representation → 11110111

SIGNED BINARY NUMBERS

- Limitation: You *must* know if number system is signed or unsigned.
  - If signed, then the left most digit is the sign bit.
  - If unsigned, then the left most digit is the most significant digit.
- Example: 01001 is $(9)_{10}$ signed or unsigned.
- Example: 11001 is $(+25)_{10}$ unsigned or $(-9)_{10}$ signed.

SIGNED BINARY NUMBERS

- Sign-magnitude → (sign bit)(magnitude bits)
- In computers, sign-complement system is more convenient for representing negative numbers.
- Negative number is indicated by its complement.
- Can use either 1's or 2's complement (2's complement is mostly used).

Signed Binary Numbers

Decimal	Signed-2's Complement	Signed-1's Complement	Signed Magnitude
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
−0	—	1111	1000
−1	1111	1110	1001
−2	1110	1101	1010
−3	1101	1100	1011
−4	1100	1011	1100
−5	1011	1010	1101
−6	1010	1001	1110
−7	1001	1000	1111
−8	1000	—	—

- All positive numbers have 0 at the left-most bit.
- Only signed - 2's complement has one representation of a 0.
- All negative numbers have 1 at the left-most bit.
- 16 binary numbers can be represented with 4 bits.
- Signed-magnitude and 1's-complement methods = Eight positive and eight negative number (including -/+ zeros)
- 2's complement method = eight positive numbers (including one zero) and eight negative numbers.

SIGNED BINARY NUMBERS ARITHMETIC

- Signed Magnitude Method
  - If the signs are the same:
    - We add the two magnitudes and give the sum the common sign.
    - Example: + 25 + 27 = + 52
    - Example: – 25 – 27 = – 52
  - If the signs are different:
    - We subtract the smaller magnitude from the larger and give the difference the sign of the larger magnitude.
    - Example: + 25 – 37 = – (37 – 25) = – 12

SIGNED BINARY NUMBERS ARITHMETIC

- Signed Complement Method
  - This method DOES NOT require a comparison or subtraction but only ADDITION.
  - The addition of two signed binary numbers with negative numbers represented in signed-2's-complement form is obtained from the addition of the two numbers, including their sign bits.
  - A carry out of the sign-bit position is discarded.

+6	00000110	–6	11111010
+13	00001101	+13	00001101
+19	00010011	+7	00000111

+6	00000110	–6	11111010
–13	11110011	–13	11110011
–7	11111001	–19	11101101

SIGNED BINARY NUMBERS ARITHMETIC

- Must ensure that the result has a sufficient number of bits to accommodate the sum.
- For two n-bit addition, if the sum occupies n+1 digits, it leads to overflow.
- Overflow is a problem in computers because the number of bits that hold a number is finite. Exceeding 1 bit will generate a wrong result.

SIGNED BINARY NUMBERS ARITHMETIC

- Take the 2's complement of the subtrahend (including the sign bit) and add it to the minuend (including the sign bit). A carry out of the sign-bit position is discarded.

$$(\pm A) - (+B) = (\pm A) + (-B);$$
$$(\pm A) - (-B) = (\pm A) + (+B).$$

- Example: (– 6) – (– 13) ➔ – 6 + 13

- 6	11111010
2's complement of (-13)	00001101
+7	100000111
After discarding the end carry:	00000111

SIGNED BINARY NUMBERS ARITHMETIC

- It is worth noting that binary numbers in the signed-complement system are added and subtracted by the same basic addition and subtraction rules as unsigned numbers.
- Therefore, computers need only one common hardware circuit to handle both types of arithmetic.
- This consideration has resulted in the signed-complement system being used in virtually all arithmetic units of computer systems.

ACTIVITY

- Consider a 5-bit computer system that used 2's complement signed binary numbers (i.e. 4 bit magnitude and 1 sign bit).
  - Convert into binary and compute (+7) + (+8). Do you get +15 in binary?
  - Now do the same with +7 + 9. Do you get +16 in binary?
  - Now do the same with (-7)+(-9). Do you get -16 in binary?
  - Now do the same with (-7)+(-10). Do you get -17 in binary?
  - State the reason for all your above observations.
  - From your observations, deduce the range of numbers in decimal that this system can use.